

DSP Engine: Audio Analysis Core

FFT, LUFS, Correlation, Clipping and Real-Time Metrics

- **FFT, LUFS, Correlation, Clipping and Real-Time Metrics**

Category: Audio Processing / DSP

Dependencies: JUCE DSP Module, FFTReal, SIMD (optional)

- **1. DSP Engine Overview**

The WhyCremisi DSP analysis engine is the AI agent's sensory system — its equivalent of the human auditory system. It operates on real-time audio buffers (64–1024 samples) and produces structured metrics that feed both the UI and the AI decision models.

◆ PROCESS FLOW

DSP ANALYZER ENGINE

Audio In → Ring Buffer → FFT Engine

→ LUFS Peak Hold

→ Phase Correlation

→ Clipping Spectrum

→ Bands Bands

ProcessAggregator

(16ms intervals)

OscBridge

→ JSON msg

Every processed audio block passes through the full analysis chain. Results are aggregated every 16ms (approximately 62 times per second at 48kHz) and sent to the presentation layer.

— 2.1 FFT ANALYSIS CHAIN

Spectral analysis is based on a Hanning-windowed FFT:

◆ OVERVIEW

FFT Parameters:

Window size 2048 samples

Hop size 512 samples (75% overlap)

Window Hanning (von Hann)

Zero-padding No

Minimum frequency 21.5 Hz (at 48kHz)

Maximum frequency 24.0 kHz (Nyquist)

Frequency resolution 23.4 Hz (per bin at 48kHz)

[NOTE] The 75% overlap guarantees a temporal resolution of ~10.6ms at 48kHz, sufficient for perceptually relevant transients while keeping computational load under control.

FFT Implementation (pseudocode):

```
void Analyzer::processFFT(const float* channelData, int numSamples) {
    // Copy into FFT circular buffer
    fftBuffer.write(channelData, numSamples);

    if (fftBuffer.available() ≥ fftSize) {
        fftBuffer.read(windowBuffer, fftSize);

        // Apply Hanning window
        for (int i = 0; i < fftSize; ++i)
            windowBuffer[i] *= hanningWindow[i];

        // Execute FFT
        fft.performRealOnlyForwardTransform(windowBuffer);

        // Compute magnitude (dB) per bin
        for (int bin = 0; bin < numBins; ++bin) {
            float real = windowBuffer[bin * 2];
            float imag = windowBuffer[bin * 2 + 1];
            magnitude[bin] = sqrtf(real * real + imag * imag);
            magnitudeDB[bin] = 20.0f * log10f(magnitude[bin] + 1e-12f);
        }
    }
}
```

— 2.2 SPECTRUM BINS

With FFT size 2048, the number of usable bins (up to Nyquist) is 1024. Each bin has a width of ~23.4Hz at 48kHz.

◆ OVERVIEW		
BIN FREQUENCY (Hz) NOTE		
0	0	DC component (discarded)
1	23.4	Extreme sub bass
2	46.9	Sub bass
... ..		
43	1007.8	Low-mids
128	3000.0	Presence
256	6000.0	High-mid
512	12000.0	Brilliance
1024	24000.0	Nyquist (discarded)

— 2.3 STEREO CHANNELS

Analysis is performed independently on both channels:

```
struct StereoAnalysis {
    float leftMagnitude[1024];
    float rightMagnitude[1024];
    float leftPhase[1024];
    float rightPhase[1024];
    float correlationValue;    // [-1.0, 1.0]
    float midMagnitude[1024]; // L+R
    float sideMagnitude[1024]; // L-R
};
```

• 3. Loudness Metrics

— 3.1 LUFS ALGORITHM (ITU-R BS.1770-4)

WhyCremisi implements loudness measurement per the ITU-R BS.1770-4 standard, with three time windows:

◆ OVERVIEW		
Type Window Update Rate		
Momentary (M)	400 ms	Every 100 ms
Short-term (S)	3 s	Every 300 ms
Integrated (I)	∞ (cumul.)	Every block

◆ PROCESS FLOW

LUFS Processing Chain:

Audio In

K-Weighting Filter

Pre-filter HP IIR: s = 38.0 Hz

(shelving)

RLB-weighting HP IIR: s = 168.1 Hz

(high-pass) 2nd order Butterworth

Mean Square per

channel $(1/N) * \sum x[n]^2$

→ →

Channel 1 Channel 2

Gain: 1.0 Gain: 1.0 (stereo — ITU weights)

Weighted sum

Loudness = $-0.691 + 10 * \log_{10}(\sum G_i * x_i^2)$

+ $10 * \log_{10}(\sum \dots)$

— 3.2 GATING

ITU-R BS.1770-4 requires a two-stage gate for Integrated LUFS calculation:

◆ OVERVIEW

Gating Algorithm:

Stage 1 — Absolute Gate:

Threshold: -70 LUFS (silence)

Blocks below threshold → excluded from calculation

Stage 2 — Relative Gate:

Compute average loudness of blocks above -70 LUFS

Relative threshold: average + 10 LUFS (typical: -25 LUFS)

Blocks below relative threshold → excluded from calculation

Result: Integrated LUFS = average of remaining blocks

[NOTE] Relative gating is crucial: without it, silence between tracks would artificially lower the integrated loudness value, leading to non-compliant broadcast masters.

— 3.3 TRUE PEAK

True Peak is calculated via 4x oversampling with sinc reconstruction filter:

True Peak = `max(|signal oversampled 4x|)`

Precision: `±0.1 dB (ITU-R BS.1770-4 compliant)`

Used for: `safe limiting, broadcast mastering`

• 4. Phase Correlation

— 4.1 PHASE CORRELATION METER

The correlation meter measures similarity between left and right channels:

◆ OVERVIEW

STEREO CORRELATION

-1.0 0.0 +0.5 +1.0



→ Out of Mono Corr.

phase compat high

CREMISI #DC143C (out of phase)

AMBER #FFB000 (mid range)

GREEN #22c55e (in phase)

Mathematical formula:

◆ OVERVIEW

$$\text{correlation} = \frac{\sum(L[n] * R[n])}{\sqrt{\sum(L[n]^2) * \sum(R[n]^2)}}$$

Interpreted values:

+1.0 → Identical (perfect mono)

+0.5 → Good mono compatibility

0.0 → Completely decorrelated

-0.5 → Evident phase problems

-1.0 → Total phase inversion

— 4.2 MID/SIDE ANALYSIS

◆ OVERVIEW

Mid (M) = L + R (everything common)

Side (S) = L - R (everything different)

M/S Ratio:

M >> S → Narrow, centered mix

M = S → Balanced, spacious mix

M << S → Wide mix, potential phase issues

— 4.3 VECTORSCOPE / PHASE SCOPE

The vectorscope is a 2D L/R display tracking instantaneous amplitudes:

◆ OVERVIEW

Left Channel

-1.0 +1.0

Right Channel

Characteristic shapes:

45° line → Perfect mono

Circle → Balanced stereo

Horizontal line → Left only (panned L)

Vertical line → 180° out of phase

• 5. Clipping Detection

— 5.1 PEAK HOLD WITH RMS

◆ OVERVIEW

CLIPPING DETECTOR

Configurable threshold: $[-\infty .. 0]$ dBFS

Default: -0.5 dBFS

Decay: 0.5 – 10 dB/s

Hold: 0 – 5000 ms

History: last 1024 events

Output:

clipDetected: bool

clipCount: int (session total)

peakValue: float (-150 .. 0 dBFS)

holdValue: float (held peak)

lastClipTime: double (samples or ms)

— 5.2 HISTORY BUFFER

```
struct ClipEvent {
    double timestamp;    // absolute sample position
    int     trackIndex;   // track ID
    int     channelIndex; // 0=L, 1=R
    float   peakValue;    // peak value in dBFS
};

class ClipHistory {
    std::array<ClipEvent, 1024> buffer;
    int writeIndex;
    int count;
    std::atomic<int> totalClips;

    void recordClip(ClipEvent event);
    const ClipEvent* getLastClip() const;
    int  getTotalClipCount() const;
    void autoReset(double timeoutMs);
    void clear();
};
```

— 5.3 AUTO-RESET

◆ OVERVIEW

Reset modes:

Manual → User clicks "reset"

Timeout → After N ms without new clips

Threshold → Clip below new threshold

Track change → Current track changes

• 6. Frequency Spectrum

— 6.1 LOG SCALE 20HZ-20KHZ

The frequency axis is mapped on a logarithmic scale to match human perception:

◆ OVERVIEW

Pixel → frequency mapping (768px display):

px freq (Hz) note

0 20 Sub bass

128 55 A1 (bass)

256 151 D3

384 415 G#4

512 1140 D#6

640 3136 G7

768 20000 Brilliance

Formula: freq = 20 * pow(20000/20, pixel / width)

— 6.2 AGGREGATE FREQUENCY BANDS

The 1024 FFT bins are aggregated into 7 musical bands for vertical bar visualization:

◆ OVERVIEW

Band Range FFT Bins UI Color

Sub 20 – 60 Hz 0 – 2 #DC143C (cremisi)

Bass 60 – 250 Hz 2 – 10 #FF6B6B

Low-Mid 250 – 500 Hz 10 – 21 #FFB000 (amber)

Mid 500 – 2 kHz 21 – 85 #FFD700

High-Mid 2 – 6 kHz 85 – 256 #22c55e (green)

Presence 6 – 10 kHz 256 – 426 #00E5FF (cyan AI)

Brilliance 10 – 20 kHz 426 – 853 #00FFaa (OK green)


```

struct FrequencyBand {
    const char* name;
    float freqMin, freqMax;
    int    binStart, binEnd;
    float energy;          // sum of magnitudes in bin range
    float peak;            // peak energy in range
    int    peakBin;        // peak bin index
    float energyDB;        // energy in dB
};

void Analyzer::computeBands(const float* magnitude,
                           FrequencyBand* bandsOut) {
    for (int b = 0; b < NUM_BANDS; ++b) {
        auto& band = bandsOut[b];
        float sum = 0.0f;
        float maxVal = -INFINITY;
        int maxBin = 0;

        for (int bin = band.binStart; bin < band.binEnd; ++bin) {
            sum += magnitude[bin];
            if (magnitude[bin] > maxVal) {
                maxVal = magnitude[bin];
                maxBin = bin;
            }
        }

        band.energy = sum;
        band.peak = maxVal;
        band.peakBin = maxBin;
        band.energyDB = 20.0f * log10f(sum + 1e-12f);
    }
}

```

— 6.3 SPECTRAL PEAK HOLD

◆ OVERVIEW

Peak Hold:

Decay rate: 2 dB/s (adjustable 0.5–10)

Initial hold: 500 ms (no decay on fresh peak)

Update: every FFT frame

Reset: on request or track change

Display:

Instantaneous spectral curve (fill)

Spectral peak hold curve (thin line)

[NOTE] Slow-decay peak hold is essential for rapid identification of problem frequencies: a momentary peak is kept visible long enough for the AI to suggest a targeted EQ intervention.

— 7.1 BUFFER MANAGEMENT

◆ PROCESS FLOW

Buffer hierarchy:

LEVEL 1: Input buffer (JUCE AudioBuffer)

Size: 64–1024 samples (DAW-variable)

Use: immediate copy → ring buffer

LEVEL 2: FFT Ring Buffer (lock-free)

Size: 4096 samples (2x FFT size)

Use: continuous accumulation for FFT window

LEVEL 3: History Ring Buffer (lock-free)

Size: 48000 samples (1s at 48kHz)

Use: headroom for delayed/LUFS analysis

LEVEL 4: ProcessAggregator (thread-safe)

Size: 64 frames (~1s of metrics)

Use: temporal smoothing, OSC/WS dispatch

— 7.2 LOCK-FREE RING BUFFER

```
template<typename T, size_t N>
class LockFreeRingBuffer {
    std::array<T, N> buffer;
    std::atomic<size_t> writePos{0};
    std::atomic<size_t> readPos{0};

public:
    bool write(const T* data, size_t count) {
        size_t wp = writePos.load(std::memory_order_relaxed);
        for (size_t i = 0; i < count; ++i) {
            buffer[(wp + i) % N] = data[i];
        }
        writePos.store((wp + count) % N,
                       std::memory_order_release);
        return true;
    }

    bool read(T* out, size_t count) {
        size_t rp = readPos.load(std::memory_order_relaxed);
        // ... mirror implementation
        return true;
    }

    size_t available() const {
        size_t wp = writePos.load(std::memory_order_acquire);
        size_t rp = readPos.load(std::memory_order_acquire);
        return (wp ≥ rp) ? (wp - rp) : (N - rp + wp);
    }
};
```

— 7.3 SIMD CONSIDERATIONS

◆ OVERVIEW

Vectorizable operations (with SIMD/AVX):

- ✓ Hanning window multiplication (vector × vector)
- ✓ Magnitude: $\text{sqrtf}(\text{real}^2 + \text{imag}^2)$ for 8 bins
- ✓ LUFS sum: mean square over 8-sample blocks
- ✓ Correlation: vectorized $L[n] \times R[n]$ product

Estimated speedup vs. scalar:

Hanning window ~4×

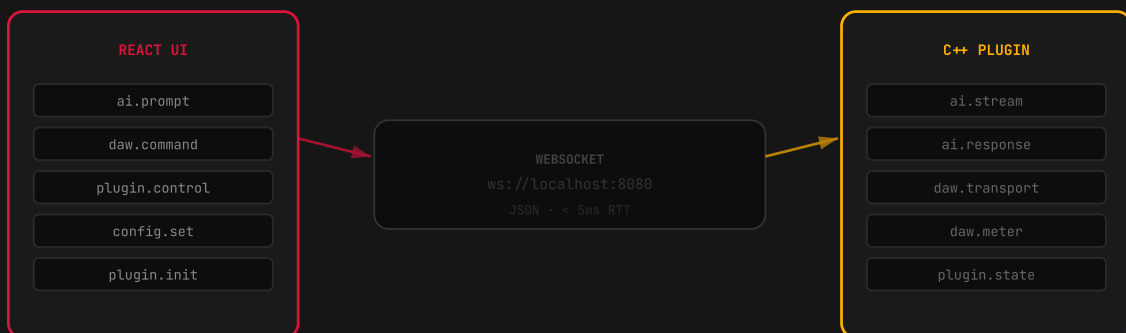
Magnitude compute ~6×

LUFS mean square ~4×

Correlation ~3×

— 7.4 THREAD SAFETY

◆ WEBSOCKET MESSAGE FLOW



[NOTE] Strict separation between audio and UI threads is critical: the audio thread must never be blocked by I/O operations or memory allocations. All communication occurs through lock-free atomic data structures.

— 8.1 PLUGINPROCESSOR CONNECTION

◆ PROCESS FLOW

PluginProcessor (JUCE)

```
processBlock(audioBuffer, midiMessages)

Analyzer::processBlock(const float** channelData,
int numChannels, int numSamples)

per channel:

processFFT(channel, numSamples)

processLUFS(channel, numSamples)

processCorrelation(channel, numSamples)

processClipping(channel, numSamples)

Analyzer::aggregate()

computeBands(magnitude, bands)

computeMeterData()
```

— 8.2 OSCBRIDGE DISPATCH

Aggregated data is serialized to JSON and sent via WebSocket:

```
{
  "type": "daw.analyzer",
  "timestamp": 1715123456789,
  "sampleRate": 48000,
  "metrics": {
    "loudness": {
      "momentary": -14.2,
      "shortTerm": -13.8,
      "integrated": -12.5,
      "truePeak": -1.2
    },
    "correlation": 0.72,
    "clipping": {
      "detected": false,
      "totalClips": 0,
      "peak": -6.3
    },
    "spectrum": {
      "bands": [
        { "name": "sub", "energy": -28.4, "peak": -22.1 },
        { "name": "bass", "energy": -18.2, "peak": -14.5 },
        { "name": "lowMid", "energy": -22.7, "peak": -18.3 },
        { "name": "mid", "energy": -16.5, "peak": -12.2 },
        { "name": "highMid", "energy": -24.1, "peak": -19.8 },
        { "name": "presence", "energy": -30.6, "peak": -26.4 },
        { "name": "brilliance", "energy": -35.2, "peak": -31.0 }
      ]
    }
  }
}
```

— 8.3 UPDATE FREQUENCIES

◆ OVERVIEW

Metric	Frequency	Max	Latency
LUFS Momentary	Every 100 ms	100 ms	
LUFS Short-term	Every 300 ms	300 ms	
LUFS Integrated	Every block	N/A (cumul.)	
True Peak	Every block	5.3 ms	
Correlation	Every 16 ms	16 ms	
Clipping	Every 16 ms	5.3 ms (1 hop)	
Spectrum bands	Every 32 ms	32 ms	
Full FFT spectrum	Every 16 ms	16 ms (full frame)	

• 9. Limitations and Future Improvements

— 9.1 CURRENT LIMITATIONS

◆ OVERVIEW

Limitation	Impact
CPU-only FFT	High CPU load
at high sample rates	
Mono/Stereo only	No multichannel
support (5.1, 7.1)	
No source separation	Rhythm overlaps
mask spectral analysis	
Fixed Hanning window	Improvable with
adaptive windows	
No ambient compensation	Background noise
unfiltered	

— 9.2 PLANNED IMPROVEMENTS

GPU Acceleration for FFT

◆ OVERVIEW

Using CUDA/Metal Performance Shaders for

concurrent FFT across multiple channels:

CPU GPU CPU

RingBuf → FFT Batch→ Magnitude

History 64 frame Aggreg.

Estimated speedup: 10–50× for batch FFT ops

Target: Integrated GPU (Apple Silicon, Intel Iris)

Machine Learning for Source Separation

◆ PROCESS FLOW

Pre-processing with lightweight model (reduced

Demucs or Spleeter) to separate signal into

components (voice, drums, bass, other) before

analysis:

Audio In → Source Separation → Analyzer per

source

→ EQ suggestion per

specific source

Multichannel Expansion

◆ OVERVIEW

Future support for:

5.1 Surround L, R, C, LFE, Ls, Rs

7.1 Surround L, R, C, LFE, Ls, Rs, Rls, Rrs

Ambisonics Spherical FFT on order 1-3

Audio objects Per-object correlation

Incremental Enhancements

- Adaptive windows (Kaiser with variable beta for resolution/spectral leakage trade-off)
- Optional zero-padding for high-resolution FFT
- Automatic noise floor detection with adaptive thresholding
- Real-time sonogram with perceptual compression (optional mel scale)
- Spectrum cache for instant A/B comparison between plugin states

• References

1. ITU-R BS.1770-4 — Algorithms to measure audio programme loudness and true-peak audio level (2015)
 2. FFTPack / JUCE DSP Module — Reference FFT implementation
 3. Zölzer, U. — Digital Audio Signal Processing (3rd ed., Wiley 2022)
 4. Smith, J.O. — Spectral Audio Signal Processing (online, 2011)
-